DO NOW — RECALL

# Lesson 074 · Big O: Constant, Linear and Logarithmic

Name: _____ Class: _____ Date: _____

**1. What is an algorithm's input size n?**

**2. What is a trace table?**

**3. What is an invariant?**

**4. A sorted search set of 128 candidates is halved repeatedly to one. How many halvings? Show the sequence.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

Number of items or other measure of problem size.

**2. Model answer**

Record of intermediate variables/states.

**3. Model answer**

Property maintained through a loop.

**4. Model answer**

128→64→32→16→8→4→2→1; seven halvings.

**By the end, you can…**

• Big O describes the upper bound on how work grows with input size, not the precise milliseconds of an execution.

• A fixed direct index operation is O(1), scanning n values is O(n), and repeatedly halving n is O(log n).

• For n=64, a linear scan may inspect 64 items, while halving 64→1 takes six halvings. Sorting requirements and constants still matter.

**Vocabulary — know these terms**

O(1) — Constant asymptotic work.

O(n) — Linear growth.

O(log n) — Logarithmic growth.

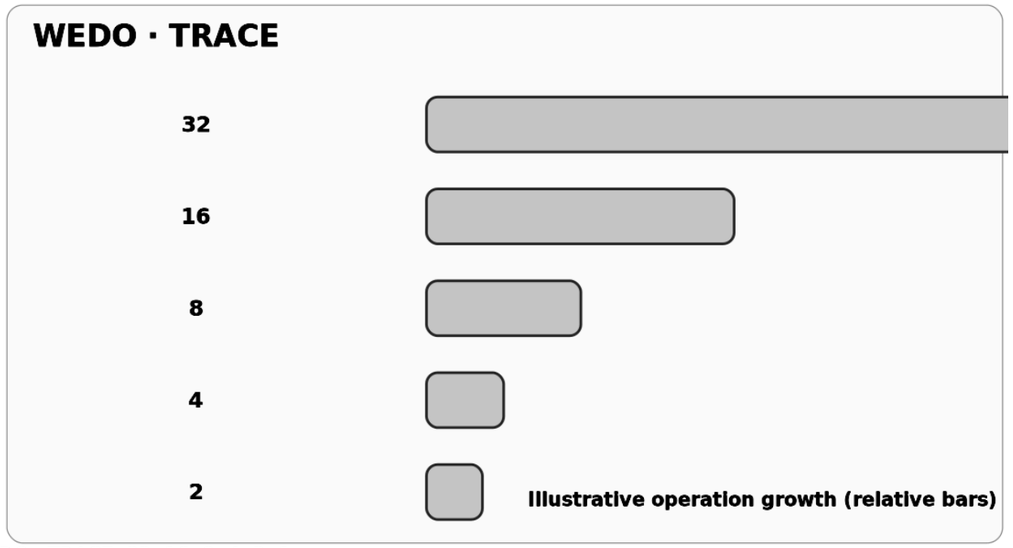
asymptotic — Behaviour as size becomes large.

# We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Classify a loop n=32; n=n/2 until n<=1.**

Each iteration cuts remaining size in half.




WEDO · TRACE: 32; 16; 8; 4; 2; 1





NEW LEARNING

# Read, annotate and explain

### Observe

Big O describes the upper bound on how work grows with input size, not the precise milliseconds of an execution.

### Explain

A fixed direct index operation is O(1), scanning n values is O(n), and repeatedly halving n is O(log n).

### Evaluate

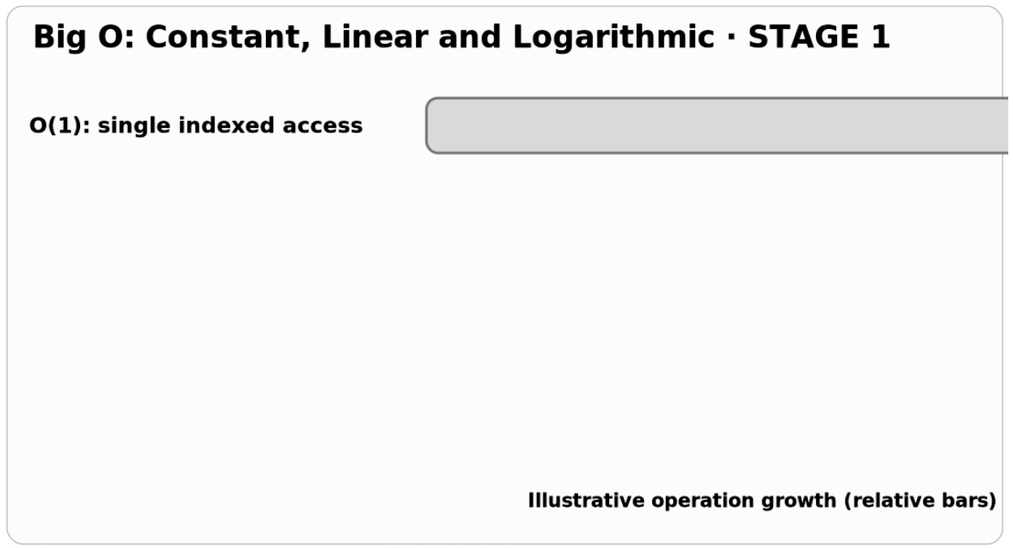
For n=64, a linear scan may inspect 64 items, while halving 64→1 takes six halvings. Sorting requirements and constants still matter.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Big O: Constant, Linear and Logarithmic · STAGE 1: O(1): single indexed access





**WE DO — CHECK AND IMPROVE**

# Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

**Guided 1 — CHECK**

Explain
Each iteration cuts remaining size in half.
Original worked example:
O(log n); values 32,16,8,4,2,1: five halvings.

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking.

**Big O directly tells you the elapsed seconds of a program.**

**Circle: Confident / Getting there / Need help**

Explain what you can now do independently.

**GREEN PEN — Correct and explain**

Choose one error. Write the correct method and explain precisely why your first response was wrong.

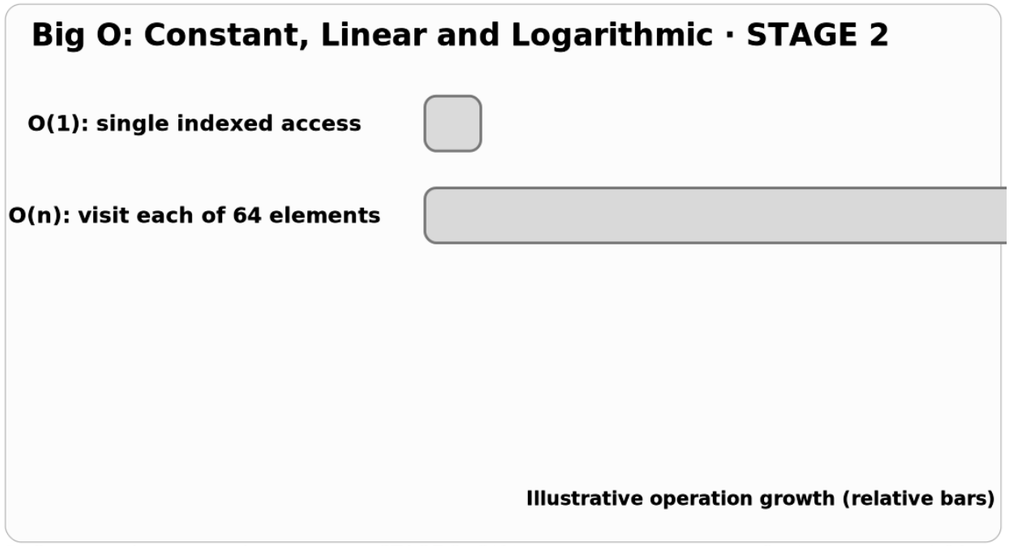




# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**

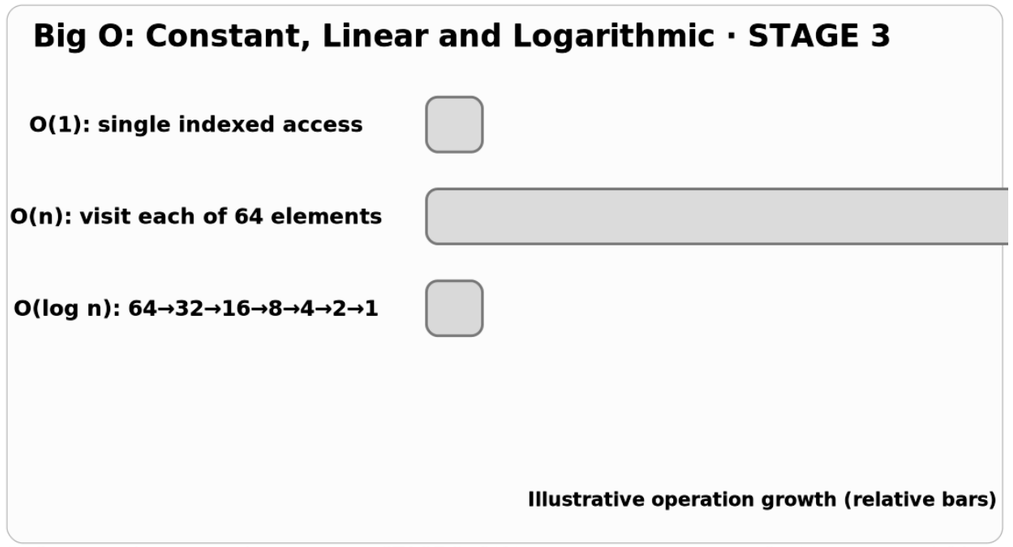



Big O: Constant, Linear and Logarithmic · STAGE 2: O(1): single indexed access; O(n): visit each of 64 elements

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Big O: Constant, Linear and Logarithmic · STAGE 3: O(1): single indexed access; O(n): visit each of 64 elements; O(log n): 64→32→16→8→4→2→1





**YOU DO — INDEPENDENT PRACTICE**

# Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

**Question 3**

Explain why O(log n) binary search may be unsuitable for unsorted data even if asymptotically faster than linear search. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

**Question 1**

Match direct array access, full array scan and repeated halving with Big O. [6]

**Question 2**

For input size64, compare scan steps64 and repeated halving to1; calculate halvings. [6]





I DO — WORKED EXAMPLE

# Follow the worked method

**Classify loop performing one action for each item in n-element array.**

**Explain**

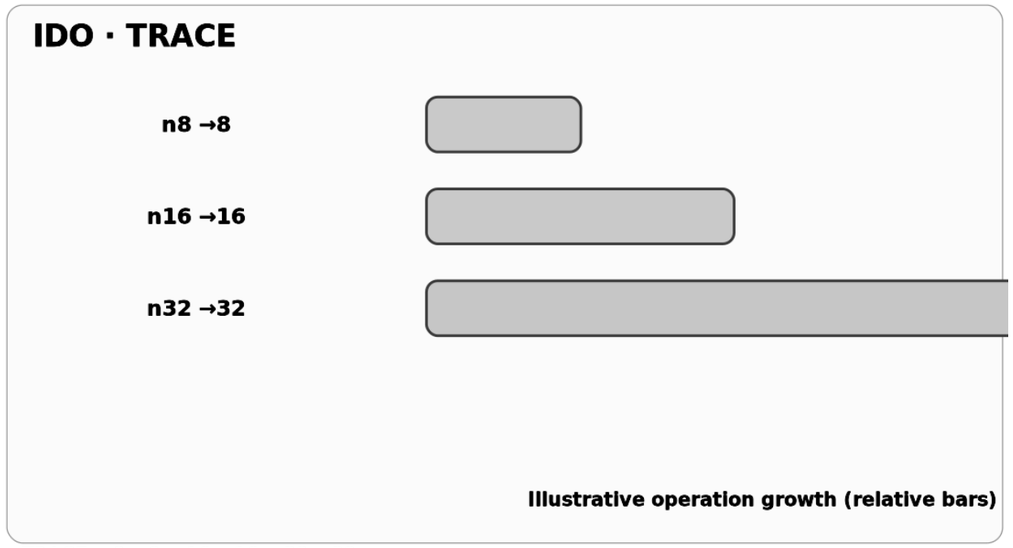
The number of operations doubles when n doubles.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**Classify loop performing one action for each item in n-element array.**

The number of operations doubles when n doubles.




IDO · TRACE: n8 →8; n16 →16; n32 →32





**WE DO — GUIDED PRACTICE**

# Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

### Guided 1 — TRY

Classify a loop n=32; n=n/2 until n<=1.